GROUP PROJECT

Web Application Development

1. Introduction

This project is designed to support the intended learning outcomes CPL02 (the ability to design a dynamic client-server web application architecture) and CPL05 (applying security principles in full-stack applications), while also training students to build systems relevant to industry needs or their own potential digital business.

Group composition: 6 groups, each with 6 members (36 students in total).

Technology stack: Vue 3 (Vite, Pinia) for the frontend; FastAPI for the backend; PostgreSQL with SQLAlchemy and Alembic for the database; Git and GitHub Actions for version control and CI/CD.

2. Topic Selection & Group Assignment

There are three system topics to choose from. Each topic is worked on by two groups in parallel (each group still builds its own system independently), so the assignment is split evenly as follows:

No	Topic	Assigned Groups
1	E-commerce Mini	1 & 2
2	Sports Venue Booking	3 & 4
3	Logistics / Delivery Tracking	5 & 6

3. Scope for Each Topic

Each topic has a set of required features that must be available by the end of the build (Session 14), as well as scope boundaries that are deliberately excluded so the project remains achievable within the available time.

3.1. E-commerce Mini

A small-scale buy-and-sell system, with a flow that goes from browsing the catalog, adding items to the cart, through to checkout.

Core data entities: User, Product, Category, Cart, CartItem, Order, OrderItem, Payment (status only), Review/Rating (optional).

Roles & access rights: Customer (shops, views order history) and Admin (manages products and orders).

Required features:

- User authentication (register/login) with JWT
- Product catalog with search and category filtering
- Shopping cart and checkout process (address form and order summary)
- Order history for the customer
- Admin dashboard: product CRUD, order status management, and a summary sales chart

Out of scope:

- ✗ Real payment gateway integration (Midtrans, Xendit, etc.) — a simulated payment status is enough
- ✗ Multi-vendor / marketplace system (multiple sellers)
- ✗ Automatic shipping cost calculation
- ✗ AI-based product recommendations
- ✗ Multi-currency support

3.2. Sports Venue Booking

A booking system for sports venues (futsal courts, badminton courts, etc.) focused on managing time-slot availability and preventing schedule conflicts.

Core data entities: Venue, Court, Schedule/Slot, Booking, Payment (deposit/paid status).

Roles & access rights: Customer (books a court) and Venue Manager (manages courts, pricing, and schedules).

Required features:

- List of venues and courts with hourly rental rates (which may differ for weekdays/weekends)
- Availability calendar for slots per court, per day
- Slot booking with conflict validation (no two bookings on the same slot and court)
- Booking confirmation (automatic once a deposit is paid, or via manager approval)
- Booking history for the customer
- Manager dashboard: court and pricing CRUD, opening/closing schedule settings, managing incoming bookings, and a summary occupancy chart

Out of scope:

- ✗ Real payment gateway integration
- ✗ Real WhatsApp/SMS notification integration (an in-app status is enough)
- ✗ Franchise / multi-city system
- ✗ Membership or loyalty point programs
- ✗ Court reviews and ratings
- ✗ Automatic dynamic pricing based on demand

3.3. Logistics / Delivery Tracking

A package delivery tracking system, with an emphasis on recording the delivery status history at each stage.

Core data entities: Shipment/Order, TrackingHistory/StatusLog, User (Customer, Courier, Admin).

Roles & access rights: Customer (creates a delivery request and tracks its status), Courier (updates the status of assigned deliveries), and Admin (assigns couriers and monitors all deliveries).

Required features:

- Customer can create a delivery request (origin-destination address, package details)
- Admin can assign a courier to each delivery
- Courier updates the delivery status through stages: Placed → Picked Up → In Transit → Delivered

- Package tracking page for the customer, showing the full status history with timestamps
- Delivery history for the customer
- Admin dashboard: list of all active deliveries, courier data management, and a summary chart of delivery volume

Out of scope:

- ✗ Real-time GPS location tracking on a map (staged status with timestamps is enough)
- ✗ Real courier-service API integration (JNE, J&T, SiCepat, etc.)
- ✗ Automatic delivery route optimization
- ✗ Complex fleet/vehicle management
- ✗ COD payment reconciliation

4. Final Report

Each group must prepare a final report, submitted together with the Session 15 presentation. The report must include:

- Background and purpose of the system.
- System design: an ERD/database schema diagram and an architecture diagram (client-server flow)
- API documentation
- Application user guide (a short user manual with screenshots of the main flows)
- Team reflection: challenges faced, key design decisions, and the division of work among members

5. Grading Rubric

The following components make up 100% of the group project grade, which itself contributes 20% to the final course grade.

Grading Component	Weight
Functionality & Feature Completeness	25%
Security (Authentication, RBAC, OWASP mitigation)	20%
Code Quality & Architecture	15%
Testing & Deployment	10%
Report Writing	10%
Presentation & Demo (Session 15)	20%
Total	100%

6. General Provisions

- Each group uses one shared GitHub repository; every member pushes using their own GitHub account so that contributions are recorded

- Progress consultations with the lecturer can be done at any time during practicum hours
- Report submission deadlines and the presentation schedule follow the academic calendar, which will be announced separately